Deal Memory

A cited, curator-checked lessons ledger so each acquisition team starts from the last deals' evidence, not from scratch

Area: M&A and post-merger integration (PMI) | Neeraj Meena | 3 November 2026

1. Area explored, and what surprised me

I chose M&A and post-merger integration as a document-heavy corner of corporate finance. A deal produces diligence reports, integration playbooks, steering-committee decks, status reports and post-mortems, and the next deal team rarely reads them. I started from an LLM-generated list of finance problems and pressure-tested the knowledge-continuity idea against this brief.

Honest fit note. Unlike tax or audit, nothing here is a legal fact, so I do not pretend the answers are exact. What can be exact is *provenance* (this claim comes from this quote on this slide) and *arithmetic* (counts across deals). The whole design is built around that split.

What surprised me. The hard part is not retrieval. It is that the evidence is thin and sanitised: a post-mortem records what people were willing to write down, and "minor delays" can hide a six-month slip. So the system must show how much evidence sits behind every answer, and sometimes say "not enough". Public disclosure confirmed it: of six SEC 8-K impairment filings I collected, three gave only an amount and a generic reason (lower expected growth, a falling share price), two gave partial causes, and only one (Versant, 2004) tied the write-down to an acquisition and named causes, including founders leaving. Filings record outcomes, not lessons.

2. The problem

Who	Corporate development and PMI teams at mid-to-large companies or private equity firms that close several deals a year, plus anyone who inherits a deal later.
Task	Before and during planning of a new acquisition, answer questions such as "In our last three software acquisitions, what caused the biggest systems-integration delays?", "Which retention approaches worked for engineering talent in similar deals?" and "Which checklist items did we miss last time?", then turn the answers into a first-draft integration checklist.
How often	Every new deal. I assume 4 to 8 a year for an active team (assumption, not validated).
Cost today	The knowledge sits in people's heads, old decks, email and shared drives. Assumption to validate: a new deal team spends one to three person-weeks rediscovering it, and people who leave take their part with them. The bigger cost is the same integration mistake repeating deal after deal, which I cannot size without interviews. I cite no deal-failure rate: commonly quoted figures define "failure" differently and I have not verified any, so the case rests on the mechanism, not a statistic.
Why now	Free-text decks and emails used to need manual tagging before they were searchable. LLMs can now draft structured records from them, and the draft can be checked mechanically (the quoted evidence must exist in the source) before a human spends time on it.
Out of scope	Live diligence on the current target (contracts, financials); tax, legal or accounting advice; valuation; real-time synergy tracking after close; culture assessment of the target; any step that acts without a person (sending, filing, deciding).
Jurisdiction	The core design is jurisdiction-neutral. What changes by country: personal-data rules for employee information in retention documents (for example GDPR in the EU, the DPDP Act in India) and limits on sharing information between deals or before closing.

3. Solution design

In, AI, out. *In:* the firm's own deal documents with their existing access rights, plus a small deal table kept by people (deal ID, date, type, size band, industry). *The AI does two jobs only:* draft structured lesson records from free text, and rank and word an answer from records that are already approved. Everything between those jobs is code or a person. *Out:* cited answers that show how many deals and documents stand behind them; a draft checklist where each item links to the lessons it came from; and a list of workstreams where the ledger has no evidence. Figure 1 (below the table) shows who acts at each step.

3.1 What must be exact, and what can be probabilistic

Part	Type	How it is kept apart and checked
Lesson text (what happened, stated cause)	Probabilistic	LLM drafts it with a verbatim quote. Code rejects it if the quote is not found. A human approves.
Quote, source location, dates, deal ID, access rights	Exact	Copied by code from the source. Never generated by the LLM.
Which deals match a question	Exact	Filter on deal metadata and the asker's rights before any ranking.
Counts and "biggest cause" rankings	Exact	SQL over approved records. The LLM cannot change a number. n is always shown.
Ranking of similar lessons	Probabilistic	Used only inside the already-filtered set; the matched record is visible to the reader.

Part	Type	How it is kept apart and checked
Answer and checklist wording	Probabilistic	Every sentence must carry a record ID. Code checks the ID exists and was retrieved. Thin evidence gives "not enough evidence".

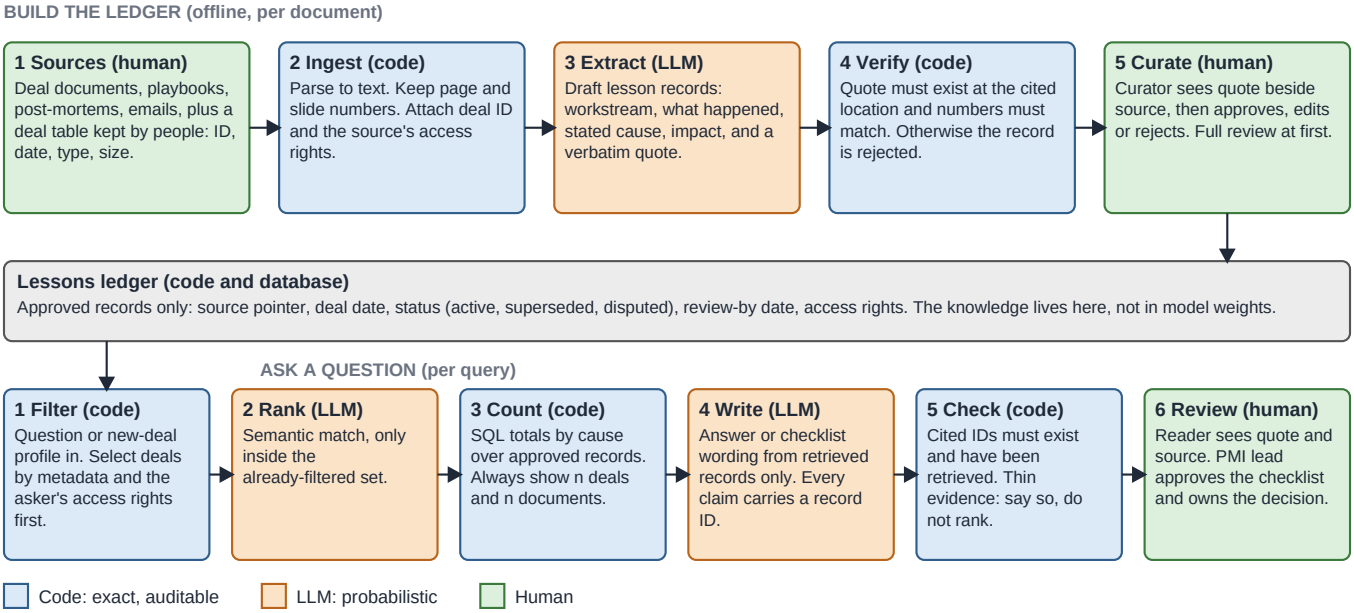

Figure 1. Who acts where. Nothing the LLM writes reaches the ledger without passing Verify and Curate, and nothing it writes reaches a reader without the Check step.

3.2 Where the knowledge lives, and when it goes stale

In the ledger (approved records with source pointers), in code (schema, rules, thresholds) and in the curators. Not in model weights: a fact in weights cannot be cited, withdrawn or access-controlled. Each record carries deal date, status (active, superseded, disputed) and a review-by date, and its age is shown beside every citation. A lesson from an older, on-premise-era integration is not silently applied to a cloud deal: the filter includes deal type and era, and a curator can mark it superseded. When two records conflict (one deal says retention bonuses worked, another says they did not), both are shown as disputed instead of being averaged.

3.3 Handling uncertainty and mistakes

- **Fabrication:** the Verify step (quote must exist) and the citation check make an unsupported record or claim fail mechanically, not by a reviewer's attention.
- **Missed lessons (recall):** a miss is invisible by nature. I would hand-label a sample of documents, measure what extraction misses, and show that recall figure to users.
- **Thin evidence:** below a threshold (for example fewer than three comparable deals) the system lists the individual cases and does not rank them.
- **Wrong output in use:** every answer has a "this is wrong" action that opens a curator ticket and marks the record disputed.

3.4 Why AI, and why not only AI

A standard post-mortem template plus a tagged wiki is the right baseline for future deals, and I would recommend it regardless. It does not help with the backlog of decks and emails written before it existed, or with questions worded differently from the tags. That is where an LLM earns its risk: turning free text into the same schema. I use it as a transcriber with a verifier, not as an oracle.

3.5 Considered and rejected

- **Chatbot over raw documents (plain RAG):** cannot count across deals, blends sources into one confident paragraph, and can leak access rights at chunk level.
- **All documents in one long context:** expensive, not reproducible, weak citations, and breaks as the archive grows.
- **Fine-tuning on deal documents:** cannot cite, cannot remove one document later, cannot respect per-user rights.
- **Several cooperating agents:** no step here gains from agents debating; it adds failure modes without adding checkable evidence.
- **Fully automatic publishing (no curator):** rejected because sources are sanitised and contain personal data. Review starts at 100 percent and drops to sampling only for low-risk workstreams, once measured precision justifies it.

3.6 The weakest point, and what would break it

The weakest point is the source, not the model. Post-mortems are written after the fact, often by people with a stake in how the deal looks, so the ledger can faithfully record a flattering story. The design cannot fix that; it can only be honest about it: each record is tagged by evidence type (measured and dated, or self-reported opinion), and answers show n and evidence

type. Two further breaks: **curator time** (if nobody reviews the queue, the ledger rots, so I would keep it small and trusted rather than large and unreviewed) and **small n** (with a handful of comparable deals, any ranking is an anecdote, hence the evidence threshold).

4. Risks and how the design handles each

Risk	How it shows up	Design response
Wrong answers	Invented or misattributed lesson; confident summary of weak evidence.	Verify step, citation check, evidence threshold, curator approval, "disputed" status, audit of extraction precision and recall.
Outdated practice	A lesson from an older deal or older tooling applied to a new context.	Deal date and age on every citation; review-by dates; superseded status; filter includes deal type and era.
Confidential data	Documents hold non-public deal information, employee pay and names, counterparties' NDAs.	Access rights carried from the source and enforced before retrieval; deal-level walls; roles instead of names in records; private deployment with no provider training on the data (contract term to confirm); query audit log; legal decides what may enter the ledger.
Liability	A team relies on the output for a retention or integration decision.	Positioned as decision support. The PMI lead stays accountable. No legal, tax or HR advice is generated; HR and legal review any retention plan. The out-of-scope list is shown in the product.
Personal data	Retention and performance documents mention individuals.	Extract roles, not names; curator checks; data-minimisation rules of the jurisdiction apply.
Model drift	A provider update changes extraction behaviour.	Versioned prompts and a fixed regression set that must pass before any change.

5. Prototype and what the tests showed

Built: a standard-library Python prototype of the pipeline: ingest, extract, verify, SQLite ledger, access filter, SQL counts and citation check (repo: github.com/Nbplanet-633/deal-memory-prototype). Real deal documents are confidential, so it runs on **13 synthetic post-mortems written for this test (AI-generated, each labelled SYNTHETIC)**: 10 main and 3 held out, with 35 hand-labelled lessons in all. There are 10 test questions with hand-worked answers, 3 of which should get "not enough evidence". Extraction was run two ways: a keyword baseline (no AI), and an LLM through OpenRouter (*nvidia/nemotron-3-ultra-550b-a55b:free*, one run).

Test	Keyword baseline	LLM
Extraction, 10 main documents (28 lessons)	Precision 92.6%, recall 89.3%	Precision 100%, recall 100%
Extraction, 3 held-out documents in other wording (7 lessons)	Precision 100%, recall 28.6% (2 of 7)	Precision 100%, recall 85.7% (6 of 7)
10 questions: counts, abstention, access filter, checklist	10/10 pass	9/10 pass
14 verifier and citation checks (invented or paraphrased quote, inflated number, wrong paragraph, uncited claim)	14/14, after one bug fix	14/14 (same code)

Where it fails. (1) Keyword rules written while reading the documents collapse on new wording (28.6% recall); the LLM held 85.7% with no wrong records kept. That is the case for using AI at the extraction step. (2) The LLM's one held-out miss was a correct governance lesson that the verifier rejected: the model attached a number of months to "dragged on for a quarter", which is not written as months. Strictness cost recall, and the design accepts that: a rejected draft can be restored by a person, but an invented number cannot be audited. (3) Verification proves provenance, not meaning. The LLM gave durations to three lessons that are not delays ("left within 6 months", "vesting over 24 months"), and the baseline tagged "kept staff attrition low" as attrition; all passed, because the quotes were real. That is why the curator stays. (4) Q8 failed under the LLM: it counted 5 deals instead of 3, because the model filed two other deals' lessons under the legal/compliance workstream. My scoring checked cause tags, not workstreams, so the error only surfaced in the answer. Fix: derive the workstream from the cause tag in code instead of asking the model. (5) The tests caught a verifier bug ("until month nine" accepted as a 9-month delay); fixed. **Limits:** one LLM run on short, clean synthetic documents; no free-text question parsing, embeddings or curator interface; real, sanitised post-mortems untested.

6. What I would check before spending more time

- **Does the data exist?** Interview 3 to 5 corporate development or PMI people (not done yet): are post-mortems written, where do they live, are they candid?
- **Size the cost.** Measure ramp-up time on the last deal; define success as time saved and mistakes not repeated.
- **Build or buy.** Check whether existing enterprise-search or deal-management tools already do the schema-and-citation part.
- **Governance first.** Ask legal what may enter the ledger (counterparty NDAs, employee data) before any ingestion.
- **Culture.** The ledger only works if teams write down failures. A post-mortem template going forward may be the cheaper first step.

Appendix: what I added beyond the AI

Everything on this page can be checked against the LLM logs submitted with this brief.

Where AI helped

- **Grok:** brainstormed finance problem areas; the M&A knowledge-continuity idea and the first problem-statement draft started there (shared chat titled "AI Ideas for Solving Finance Problems"; full prompt in the logs).
- **Claude:** read the take-home brief, tested the idea against it (public data, exact versus probabilistic split, why not a simpler approach), drafted this document, and wrote the prototype code, the synthetic data and the tests. Key prompts: (1) I pasted Grok's problem statement together with the assignment PDF; (2) "with my M&A problem make the entire tresata pdf"; (3) "make me prototype aslo using data like you mentioned above".

What I rejected or corrected

- Grok's draft said studies consistently show a large share of deals miss their thesis largely because of integration, with no source. Removed; this brief makes no failure-rate claim.
- Grok's draft proposed a knowledge base that answers questions like "what caused the biggest delays across our last three deals" without saying how cross-deal comparison stays reliable. Added: such questions need counting across records, so counts come from code over approved records, not from the LLM summarising text.
- Claude advised that this idea was a weak fit (no public internal data, no exact legal core) and suggested a tax-document problem instead. I kept M&A and tested its weak points instead of switching: I asked for real public data and a working prototype, so the two weaknesses (no public internal data, no exact legal core) became stated and measured limits (Sections 3.6 and 5) instead of hidden ones.
- The first AI-written verifier accepted a claimed 9-month delay because the quote said "until month nine"; the adversarial tests caught it and it was fixed. The AI's keyword extractor also looked strong at first (92.6% precision) only because its rules were written against the same documents; the held-out set in new wording dropped its recall to 28.6%.

What came from me

- **Problem choice:** keeping M&A post-merger integration after being advised to switch to a tax problem with a stronger public-data fit. The cost I accepted: the prototype can only be tested on synthetic documents.
- **Real data over invented examples:** I pushed for actual public filings rather than only made-up ones. That showed filings report outcomes, not causes (Section 1), which became the weakest-point argument in 3.6.
- **Build, not just describe:** I chose to build and test the prototype instead of stopping at the design, which is how the verifier bug, the keyword extractor's weakness on new wording and the LLM's workstream error (Q8) were found. I ran the LLM evaluation on my own machine.

One thing I would stand behind without AI

The split between what code does and what the model does (Section 3.1). The quote, the deal and paragraph it came from, the counts across deals and the access filter are all code, because they must be exact and checkable: if the model produced a number or a source, nobody could audit it. The model only drafts lesson records and wording, and everything it drafts is checked by code (the quote must exist verbatim, every cited record must have been retrieved) and approved by a person before anyone relies on it.

LLM logs are submitted separately as a zip of the chats (Grok conversation and Claude conversation).